AULA 06 - FUNÇÃO MAIN E DIRETIVAS

Profª. Drª. Tatiele B. Moro

Função main

- A lógica do programa é implementada por uma função principal, denominada `main()`

- Todo programa em C deve ter uma função `main()`, por onde se inicia a sua execução.

- Os comandos da função são delimitados por chaves.

Retorno da função main

- A função main pode retornar um inteiro para o sistema operacional

- Para isso utilizamos

```
int main()
```

- e podemos utilizar o comando `return` para encerrar a função main e passar um inteiro para o sistema operacional

- O retorno zero indica ao SO que o programa foi bem sucedido. Cada outro número retornado indica o código de uma condição de erro.

Parâmetros da função main

- O protótipo da função **`main`** depende do sistema operacional subjacente:
- // padrão C ANSI
  - int main (void) ;
  - int main (int argc, char **argv) ;
  - int main (int argc, char *argv[]);
- // sistemas UNIX-like (Linux, FreeBSD, Solaris, ...) e Windows
  - int main (int argc, char **argv, char **envp) ;
- // sistemas Apple (MacOS, iOS, ...)
  - int main (int argc, char **argv, char **envp, char **apple) ;

Parâmetros da função main

- Significado dos parâmetros usuais:

  - “argc”': Um inteiro que contém a contagem dos argumentos que seguem em argv. O parâmetro argc é sempre maior ou igual a 1.

  - “argv”: Uma matriz de cadeias de caracteres terminadas em nulo que representam argumentos de linha de comando inseridos pelo usuário do programa. Por convenção, argv[0] é o comando com que o programa é invocado. argv[1], é o primeiro argumento da linha de comando. O último argumento da linha de comando é argv[argc - 1], sendo que argv[argc] é sempre NULL.

Exemplo

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    int i;
    for (i = 0; i < argc; i++)
        printf("%s\n", argv[i]);
    return 0;
}
```

Diretivas do pré-processador

- Permite que o programador modifique a compilação
- O pré-processador é um programa que examina e modifica o código fonte antes da compilação
- As diretivas são os comandos utilizados pelo pré-processador
  - Estes comandos estarão disponíveis no código fonte, mas não no código compilado
- As diretivas iniciam com #
  - Ex.: #include

Diretiva #include

```
#inclute<nome_do_arquivo>
```

- As diretivas servem para ordenar o pré-processador para carregar o conteúdo do arquivo de texto `nome_do_arquivo`, como parte integrante.

- Além das bibliotecas padrão, podemos chamar nossos próprios arquivos em diretivas para evitar digitação.

Diretiva #include

- Portanto, permite inserir um arquivo qualquer no código-fonte
- A diretiva include é substituída pelo conteúdo do arquivo
- Quando usamos <> para indicar o arquivo, este arquivo é procurado somente na pasta include (no linux, /usr/include)
- Quando utilizamos “” para indicar o arquivo, este arquivo é procurada na pasta atual, e se não for encontrado é procurado na pasta include.

Diretiva #include

```
#include <stdio.h>
```

- As bibliotecas padrão recebem uma extensão .h de head (cabeçalho).

- As rotinas contidas nestas bibliotecas são reconhecidas, porque os arquivos `.h` são programas em C

Diretiva #include

- Exemplo:

```
int soma(int a, int b)
{
    return a+b;
}
```

arquivo.c

```
#include "arquivo.c"

int main()
{
    soma(1, 2);
    return 0;
}
```

principal.c

Diretiva #define

- Permite definir constantes sem consumir memória durante a execução
- Não use o sinal de atribuição (=) !

```
#define PI 3.14
int main()
{
    double raio = 1.0;
     double area = PI * raio * raio;
     return 0;
}
```

Diretiva #define

- Permite definir trechos fixos de código

```
#include <stdio.h>
#define ERRO printf("Ocorreu um erro\n");
int main()
{
      ERRO;
      return 0;
}
```

Diretiva #define

- Permite definir trechos de código com parâmetros (macros)
- Não pode ter espaços no identificador.
  - Ex.: SOMA (x,y)

```
#define SOMA(x,y) x + y
int main()
{
    int a = SOMA(1, 2);
    double b = SOMA(1.0, 2.0);
    return 0;
}
```

Diretiva #define

- Recomenda-se usar parênteses em macros.

```
#include <stdio.h>
#define SOMA(x,y) x + y
int main()
{
    printf("%d\n", 10 * SOMA(1,2));
    return 0;
}
```

Diretiva #define

- Recomenda-se usar parênteses em macros.

```
#include <stdio.h>
#define SOMA(x,y) x + y
int main()
{
    printf("%d\n", 10 * SOMA(1,2));
    return 0;
}
```

// solução: #define SOMA(x,y) **(**x + y**)**

Diretiva #define

- Recomenda-se usar parênteses em macros.

```
#include <stdio.h>
#define PRODUTO(x,y) (x * y)
int main()
{
    printf("%d\n", PRODUTO(2+3, 4));
    return 0;
}
```

Diretiva #define

- Recomenda-se usar parênteses em macros.

```
#include <stdio.h>
#define PRODUTO(x,y) ((x) * (y))
int main()
{
    printf("%d\n", PRODUTO(2+3, 4));
    return 0;
}
```

// solução: #define PRODUTO(x,y) **((x) * (y))**

Diretiva #undef

- Remove a definição criada com #define

```
#define TAM_STRING 20
...
#undef TAM_STRING
#define TAM_STRING 100
```

Compilação condicional

```
#include <stdio.h>
#define DEBUG
int main()
{
    #ifdef DEBUG
        printf("Descricao detalhada: ...\n");
    #else
        printf("Nenhuma descricao\n");
    #endif
}
```

Compilação condicional

```
#include <stdio.h>
#define DEBUG 0
int main()
{
    #if DEBUG == 1
        printf("Descricao detalhada: ...\n");
    #elif DEBUG == 2
        printf("Descricao resumida: ...\n");
    #else
        printf("Nenhuma descricao\n");
    #endif
}
```